Compte rendu

Module : Méthodes Agile

Niveau : M1 MIAGE

Fait par :

YAHIAOUI AMIRA

et

YANIS BENAMOR

Professeur :

Hammoudi Karim

18 mars 2026

Table des matières

1 Introduction **2**

2 Objectifs du projet **2**

3 Architecture technique et choix technologiques **2**

4 Réalisations marquantes **3**

4.1 Algorithme Peak Demand Alert 3

4.2 Assistant IA contextuel 3

5 Fonctionnalités développées par l'équipe **3**

5.1 Gestion dynamique de l'inventaire et des variantes 3

5.2 Traçabilité des mouvements de stock 3

6 Défis techniques rencontrés et solutions apportées **3**

6.1 Conteneurisation et gestion des CORS 3

6.2 Gestion des URLs des médias 4

6.3 Gestion des cas sans historique 4

7 Procédure d'installation **4**

7.1 Configuration 4

7.2 Lancement 4

7.3 Initialisation de la base de données 4

7.4 Accès 4

8 Diagrammes de Classes **4**

8.1 Diagramme de la base de données 5

8.2 Diagramme de l'application 5

9 Conclusion **6**

1. Introduction

Le projet **StockFlow** a été conçu pour répondre aux problématiques modernes de la gestion d'inventaire en environnement B2B. Face à la nécessité croissante de réactivité dans les chaînes d'approvisionnement, notre équipe a développé une application web de type SaaS (*Software as a Service*) permettant un suivi en temps réel des stocks, couplé à des outils d'analyse prédictive et d'assistance par intelligence artificielle.

Ce document présente l'architecture mise en place, les fonctionnalités développées, ainsi que les principaux défis techniques rencontrés lors du développement.

2. Objectifs du projet

L'objectif principal de StockFlow est de fournir une interface centralisée permettant aux gestionnaires de stocks de :

- suivre les niveaux d'inventaire avec précision, notamment grâce à une gestion par variantes de produits ;
- anticiper les ruptures de stock à l'aide d'algorithmes d'analyse des ventes ;
- interagir avec les données de manière intuitive via un assistant IA conversationnel intégré ;
- garantir un déploiement robuste, reproductible et adaptable à différents environnements.

3. Architecture technique et choix technologiques

Dans cette section, nous présentons les choix techniques retenus pour la conception du système.

- **Frontend :** développé en React.js avec Tailwind CSS pour une interface modulaire et réactive ;
- **Backend :** développé avec le framework Laravel, exposant une API REST ;
- **Base de données :** MySQL, garantissant la cohérence des données relationnelles ;
- **Infrastructure :** Docker et Docker Compose, avec Nginx comme reverse proxy.

4. Réalisations marquantes

4.1. Algorithme Peak Demand Alert

Afin d'améliorer la pertinence fonctionnelle du système, un module d'alerte de pic de demande a été développé. Cet algorithme analyse les mouvements de stock des dernières 48 heures et les compare à une période précédente afin de détecter les variations significatives.

4.2. Assistant IA contextuel

Un assistant basé sur un modèle de langage (LLM, *Large Language Model*) a été intégré dans l'application. Celui-ci communique avec le backend via API et permet d'interroger les données de l'inventaire de manière intuitive.

5. Fonctionnalités développées par l'équipe

5.1. Gestion dynamique de l'inventaire et des variantes

L'application permet la création et la gestion de produits comportant plusieurs variantes (taille, couleur, etc.). Les images associées sont téléversées et stockées de manière sécurisée.

5.2. Traçabilité des mouvements de stock

Chaque modification du stock génère un enregistrement dans une table dédiée, permettant de conserver un historique fiable et exploitable pour les analyses.

6. Défis techniques rencontrés et solutions apportées

6.1. Conteneurisation et gestion des CORS

Le passage à une architecture Docker a entraîné des problèmes de communication entre le frontend et le backend.

Solution : pour résoudre ce problème, nous avons mis en place un reverse proxy ainsi qu'une configuration adaptée des règles CORS côté Laravel.

6.2. Gestion des URLs des médias

Des erreurs 404 apparaissaient en raison d'une mauvaise génération des URLs des images.

Solution : nous avons configuré la génération d'URLs absolues à partir de la variable `APP_URL` et corrigé les données existantes en base.

6.3. Gestion des cas sans historique

L'algorithme de détection provoquait une erreur en cas d'absence d'historique de données.

Solution : une logique conditionnelle a été ajoutée afin d'éviter les divisions par zéro et garantir le bon fonctionnement de l'application.

7. Procédure d'installation

7.1. Configuration

- cloner le dépôt Git ;
- configurer les fichiers `.env` ;
- ajouter la clé API d'intelligence artificielle.

7.2. Lancement

```
docker compose up -d --build
```

7.3. Initialisation de la base de données

```
docker compose exec backend php artisan migrate:fresh --seed
```

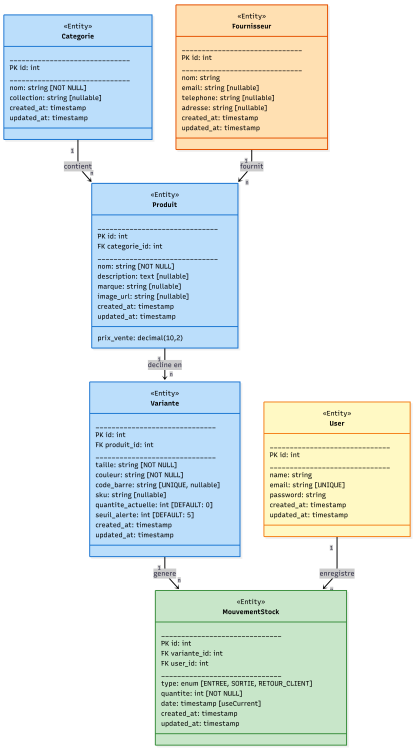
7.4. Accès

`http://localhost:3000`

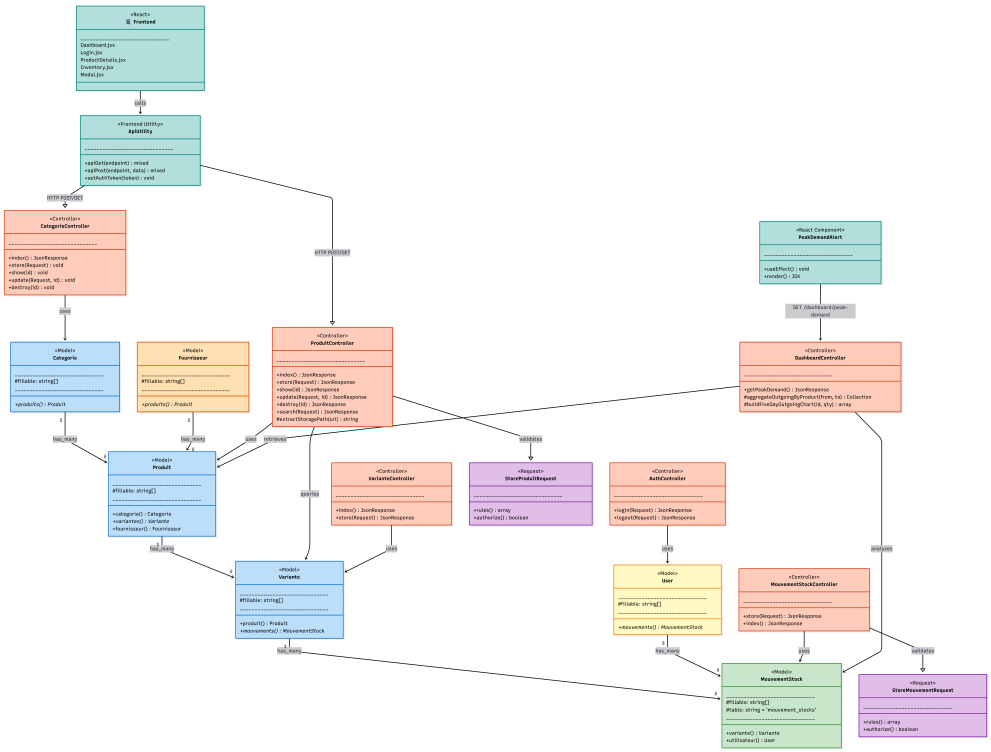
8. Diagrammes de Classes

Cette section présente les diagrammes de classes du projet, illustrant la structure de la base de données ainsi que l'architecture de l'application.

8.1. Diagramme de la base de données



8.2. Diagramme de l'application



9. Conclusion

Le projet StockFlow a permis de concevoir une architecture complète et moderne, allant de la base de données jusqu'au déploiement. Les défis techniques rencontrés ont été résolus de manière rigoureuse, aboutissant à une application stable et fonctionnelle.

Ce projet constitue une base solide pour des évolutions futures, notamment en matière d'analyse prédictive et d'intégration avancée de l'intelligence artificielle.